

Fermi Estimate: GPU Fleet for a Chat LLM at 100M DAU

Practice date: 2026-10-03. Problem-solving type: puzzles & estimation. Worked interactively, interviewer-style.

The question

Estimate the GPU fleet, in H100-equivalents, needed to serve a chat LLM product with 100M daily active users at ChatGPT-style usage.

How to approach it (the method)

Fermi interviews grade decomposition, stated assumptions, and sanity-checking — not the final number. The key structural insight, stated up front:

> The GPU fleet is a **tokens-per-day** question. API fan-out, databases, and caching shape latency architecture; they are second-order for fleet size. Caching is just a discount factor on tokens at the end.

The whole estimate collapses to one equation:

GPUs = daily tokens / (tokens/sec per GPU x 86,400 x utilization), then x peak-to-average factor

with daily tokens = DAU x sessions per user x turns per session x tokens per turn.

An H100 is NVIDIA's datacenter GPU, the standard unit of AI compute (80GB high-bandwidth memory). For a frontier-class chat model with batching, assume on the order of a few hundred to ~1k tokens/sec per H100 — stated as an assumption, not derived.

First attempt and corrections

Initial guesses: 10 sessions/day, 5-6 turns per session, 10-12k tokens per turn. Both activity and token assumptions failed reality checks:

- 10 sessions x 5.5 turns = 55 turns/user/day. At a minute per turn, that's nearly an hour of pure chatting daily for the *average* of 100M users — power-law reality says most DAUs do one or two quick sessions.
- 10-12k tokens per turn (~7,500-9,000 words) is enterprise/RAG territory, borrowed from coding-assistant usage. A chat turn is a paragraph in, a few paragraphs out — hundreds of tokens.

Revised assumptions: 5-6 sessions/day, 2 turns per session, 300-400 tokens per turn.

The arithmetic

- Daily turns: $100\text{M} \times 5.5 \times 2 = \mathbf{1.1B \text{ turns/day}}$ (~12,700/sec average)
- Daily tokens: $1.1\text{B} \times 350 = \mathbf{\sim 385B \text{ tokens/day}}$
- Per GPU: $500 \text{ tok/s} \times 86,400 \times 0.5 \text{ utilization} = \mathbf{\sim 21.6M \text{ tokens/GPU/day}}$

- Average fleet: $385\text{B} / 21.6\text{M} = \sim\mathbf{18k\ H100s}$
- Peak-to-average $\sim 2.5\times$ (chat is spiky across timezones): $\sim\mathbf{40-50k\ H100s}$

Sanity anchor: public reporting puts OpenAI's total fleet in the hundreds of thousands of GPUs — but that includes training and a far larger user base. Tens of thousands for inference-only at 100M DAU passes the smell test. Anchoring the output against a known quantity is the move that scores.

Follow-up: 40% of traffic shifts to long-context agentic sessions

Qualitative calls (all correct): fewer human turns per session, much larger tokens per turn, agents become the traffic source instead of humans, and peaks flatten because agentic work is deferrable/async.

Re-running the equation:

- 40% of 1.1B turns = 440M agentic turns. One human turn triggers an agent run: ~ 10 model calls (plan, tool calls, synthesis) $\times \sim 2\text{k}$ tokens each = $\sim\mathbf{20k\ tokens\ per\ agentic\ turn}$ vs. 350 before ($\sim 50\times$; conservative $10\times$ still dominates).
- Agentic slice: $\sim 154\text{B}$ tokens/day $\rightarrow \sim 1.5\text{T}$. Total daily tokens $\sim 385\text{B} \rightarrow \sim\mathbf{1.5-2T\ (roughly\ 4\times)}$.
- Second-order effect: long context also **lowers** per-GPU throughput — large KV-caches force smaller batches, so tokens/sec/GPU degrades exactly when you need more tokens. It compounds.
- Partial offset: async agents run off-peak, so the peak factor drops ($\sim 2.5\times \rightarrow \sim 1.8\times$).

Net: average fleet $\sim 4\times$ to $\sim 70\text{k}$, $\times 1.8$ peak = $\sim\mathbf{120-150k\ H100s}$, up from $\sim 45\text{k}$. The headline: the 40% agentic slice ends up dominating the entire fleet.

Key takeaways

- Lead with the dominant term. Say "this is a tokens-per-day question; everything else is second-order" in the first 30 seconds.
- Sanity-check every assumption against your own usage before the interviewer has to prompt you. 55 turns/day should feel wrong the moment you say it.
- Split input vs. output tokens when you can: prefill (input) is parallel and cheap; decode (output) is sequential and is the real GPU bottleneck. Naming the split shows serving literacy.
- Never end a Fermi follow-up with "about the same" — re-run your own equation with the changed variable, and always ask whether your throughput assumption survives the change.